ClearQuote SDK – React Native Integration Guide

1. Introduction

This document describes how to integrate the ClearQuoteSDK into a React Native application using a native module bridge. The SDK enables vehicle inspection flows, including camera capture, offline inspections, and inspection completion callbacks.

Supported platforms:

- iOS — Integrated (ClearQuoteSDK via Swift Package Manager + ClearQuoteModule.swift)
- Android — Not Implemented yet

2. Prerequisites

Before starting the integration, ensure the following requirements are met:

Common

- Node.js: 22.11+
- React Native: 0.76+
- A valid ClearQuote SDK Key (contact the ClearQuote team)

2.1 iOS

- Supported Device: iPhone
- Minimum iOS Deployment Target: iOS 16.0
- Xcode Version: 16.1 or later
- Swift Package Manager support in the iOS project
- Swift support must be enabled in the iOS project

2.2 Android

Not Implemented yet

Android ClearQuoteSDK React Native bridging is not available yet. Android-specific prerequisites will be documented when support is added.

3. Architecture

React Native talks to ClearQuoteSDK through a native module and an event emitter:

```
React Native (JS / TypeScript)
    ↕ NativeModules.ClearQuoteModule
    ↕ NativeEventEmitter   inspectionCompletionStatus
    ├─ iOS     → ClearQuoteModule.swift → ClearQuoteSDK (SPM)
    └─ Android → Not Implemented yet
```

4. Install ClearQuote SDK

4.1 iOS

Step 1: Add ClearQuoteSDK via Swift Package Manager

In Xcode, open ios/MyApp.xcworkspace, then:

- File → Add Package Dependencies…
- Add Repository URL: https://github.com/clearquotetech/cq-ios-sdk.git

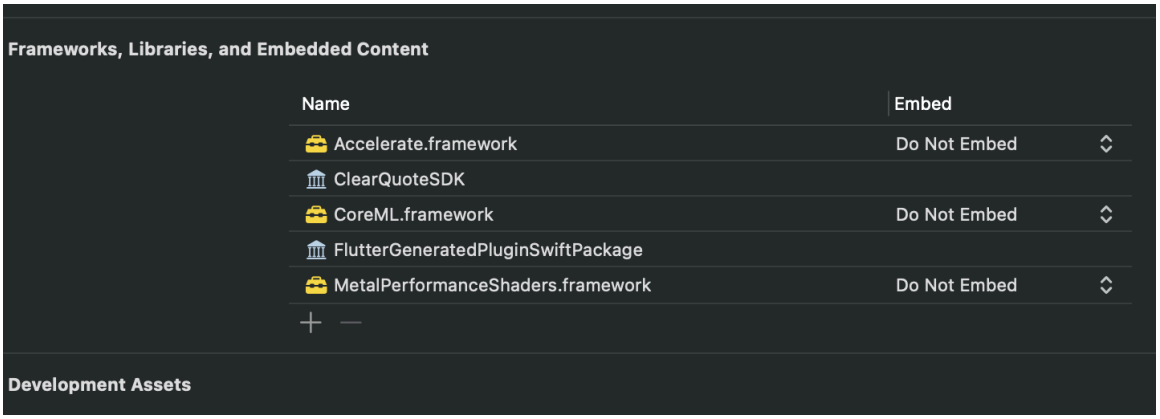
```swift
.package(
    url: "https://github.com/clearquotetech/cq-ios-sdk.git",
    .branch("main")
)// Xcode 27 & above

.package(
    url: "https://github.com/clearquotetech/cq-ios-sdk.git",
    .branch("xcode26")
)// Xcode 26 (Supporting till End of 2026)
```

- Add the ClearQuoteSDK product to your app target

Also link these Apple frameworks on the app target for SDK runtime:

- CoreML
- MetalPerformanceShaders
- Accelerate



Step 2: Add the SVGKit ModuleShim

This step is required for React Native iOS. React Native uses static frameworks, and ClearQuoteSDK's Swift interface still imports SVGKit. SVGKit from SPM does not produce a real framework module, so the build fails unless you add a compile-time shim. Runtime symbols still come from the SPM SVGKit target.

Note: ModuleShims is available in demo app

Create these files under ios/ModuleShims/SVGKit.framework/:

ios/ModuleShims/SVGKit.framework/Headers/SVGKit.h

```
#import <Foundation/Foundation.h>

// Compile-time shim so ClearQuoteSDK's .swiftinterface can `import
SVGKit`
// when SVGKit is linked statically via SPM (no real framework
product).
// Runtime symbols still come from the SPM SVGKit target via
ClearQuoteSDKDeps.
```

ios/ModuleShims/SVGKit.framework/Modules/module.modulemap

```
framework module SVGKit {
  umbrella header "SVGKit.h"
  export *
}
```

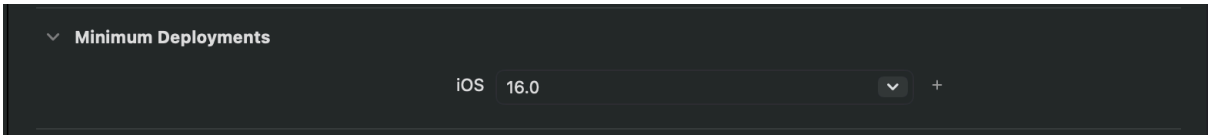
Then add the shim directory to Framework Search Paths (Debug and Release) on the app target:

```
FRAMEWORK_SEARCH_PATHS = (
  $(inherited),
  $(SRCROOT)/ModuleShims
```

```
)
```

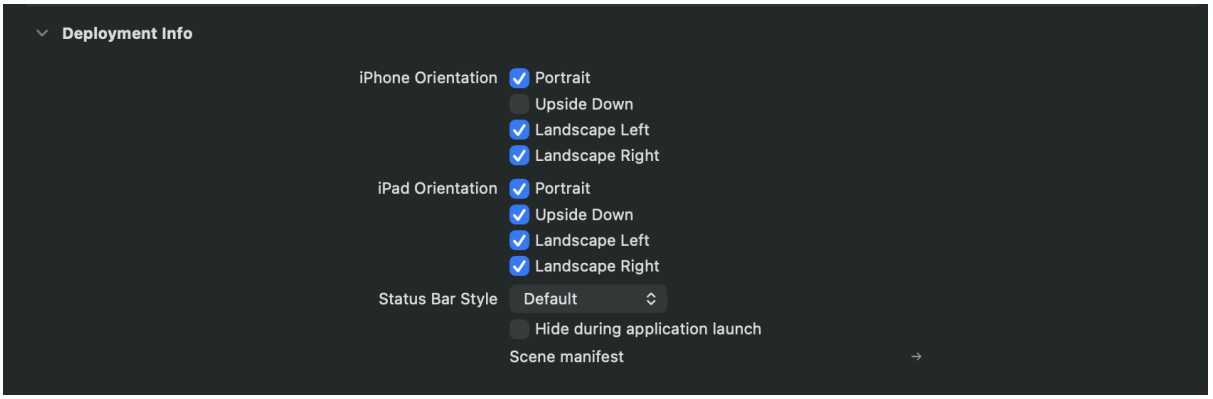
Step 3: Set iOS Deployment Target

Ensure IPHONEOS_DEPLOYMENT_TARGET is set to 16.0 for the app project and targets.



Step 4: Add supported interface orientation

SDK required below supported interface orientation



Step 5: Update Info.plist

- Camera permission

```xml
<key>NSCameraUsageDescription</key>
<string>For capturing vehicle images</string>
```

- Location permission

If using location feature from the SDK then based on **locationUsage** (**foregroundOnly** OR **withBackground**) selected, set the relevant info.plist keys

```swift
ClearQuote.shared.locationUsage = .foregroundOnly/.withBackground
// Set this in didFinishLaunchingWithOptions of AppDelegate if want to use location feature. By default, locationUsage is disabled

// foregroundOnly - location is used while using the app
// withBackground - location is used while using the app & also when the app is in background
```

- If **locationUsage** is **foregroundOnly**

Key: Privacy - Location When In Use Usage Description

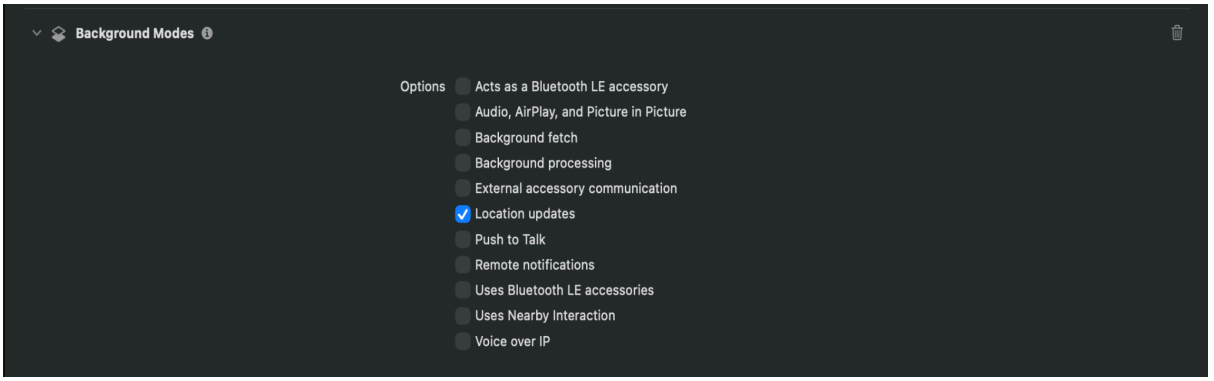
Value: <your app name> needs your location while using the app to capture inspection locations

- If **locationUsage** is **withBackground**

Key: Privacy - Location Always and When In Use Usage Description

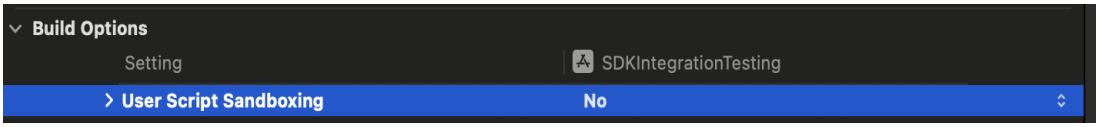
Value: <your app name> needs your location while using the app to capture inspection locations and sync your inspections

Add Background modes support with location updates in the source app if choosing **locationUsage** as **withBackground**



Step 6: Disable User Script Sandboxing

Make sure app sandboxing is off in build options of the target app.



Step 7: Add Run Script

Add the following Run Script on the app target so the app can be uploaded to the App Store. Keep it after Embed Frameworks.

```
set -euo pipefail

VERSION="1.0.0"
MIN_OS="${IPHONEOS_DEPLOYMENT_TARGET}"
```

```
patch_plist() {
  plist="$1"
  if [ ! -f "${plist}" ]; then
    return 0
  fi

  if /usr/libexec/PlistBuddy -c "Print :CFBundleShortVersionString"
"${plist}" >/dev/null 2>&1; then
    /usr/libexec/PlistBuddy -c "Set :CFBundleShortVersionString
${VERSION}" "${plist}"
  else
    /usr/libexec/PlistBuddy -c "Add :CFBundleShortVersionString
string ${VERSION}" "${plist}"
  fi

  if /usr/libexec/PlistBuddy -c "Print :MinimumOSVersion" "${plist}"
>/dev/null 2>&1; then
    /usr/libexec/PlistBuddy -c "Set :MinimumOSVersion ${MIN_OS}"
"${plist}"
  else
    /usr/libexec/PlistBuddy -c "Add :MinimumOSVersion string
${MIN_OS}" "${plist}"
  fi

  echo "Patched ${plist}"
}

# 1) Patch SourcePackages artifacts so future embeds already have
valid keys.
DERIVED_DATA_ROOT="${BUILD_DIR%%/Build/*}"
ARTIFACTS_ROOT="${DERIVED_DATA_ROOT}/SourcePackages/artifacts/tensor
flowlitec"
if [ -d "${ARTIFACTS_ROOT}" ]; then
  find "${ARTIFACTS_ROOT}" -path
"*/TensorFlowLiteC*.framework/Info.plist" | while IFS= read -r
plist; do
    patch_plist "${plist}"
  done
else
  echo "note: TensorFlowLiteC SPM artifacts not found at
${ARTIFACTS_ROOT}"
fi

# 2) Patch (and re-sign) frameworks already embedded in the app
bundle.
FRAMEWORKS_DIR="${TARGET_BUILD_DIR}/${FRAMEWORKS_FOLDER_PATH}"
if [ -d "${FRAMEWORKS_DIR}" ]; then
```

```
  for name in TensorFlowLiteC TensorFlowLiteCMetal
TensorFlowLiteCCoreML; do
    fw="${FRAMEWORKS_DIR}/${name}.framework"
    patch_plist "${fw}/Info.plist"
    if [ -d "${fw}" ] && [ -n "${EXPANDED_CODE_SIGN_IDENTITY:-}" ]
&& [ "${EXPANDED_CODE_SIGN_IDENTITY}" != "-" ] && [
"${CODE_SIGNING_ALLOWED:-YES}" != "NO" ]; then
      /usr/bin/codesign --force --sign
"${EXPANDED_CODE_SIGN_IDENTITY}"
--preserve-metadata=identifier,entitlements,flags --timestamp=none
"${fw}"
    fi
  done
else
  echo "note: Frameworks directory not found at ${FRAMEWORKS_DIR};
skipping embedded patch"
fi
```

4.2 Android

Not Implemented yet

Android ClearQuoteSDK installation for React Native is not available yet. Installation steps will be documented when Android support is added.

5. Native Bridge Setup

5.1 iOS

Create a native module that subclasses RCTEventEmitter and calls ClearQuoteSDK APIs.

Step 1: Create ClearQuoteModule.swift

Add ios/ClearQuoteModule.swift. Core responsibilities:

- Native module name: ClearQuoteModule
- Event: inspectionCompletionStatus
- Map JS calls to ClearQuote.shared APIs
- Forward ClearQuoteSDKDelegate.inspectionCompletionStatus to the event emitter

Example skeleton:

```
import Foundation
import ClearQuoteSDK
import UIKit
import React
```

```
@objc(ClearQuoteModule)
class ClearQuoteModule: RCTEventEmitter {

  private var hasListeners = false

  override func supportedEvents() -> [String]! {
    ["inspectionCompletionStatus"]
  }

  override func startObserving() { hasListeners = true }
  override func stopObserving() { hasListeners = false }

  @objc(initSDK:resolver:rejecter:)
  func initSDK(
    _ key: String,
    resolver resolve: @escaping RCTPromiseResolveBlock,
    rejecter reject: @escaping RCTPromiseRejectBlock
  ) {
    DispatchQueue.main.async {
      guard let rootVC =
UIApplication.shared.topMostViewController() else {
        reject("NO_VC", "No ViewController", nil)
        return
      }
      ClearQuote.shared.initSDK(baseVC: rootVC, key: key) {
isInitialized, code, message in
        resolve(["isInitialized": isInitialized, "code": code,
"message": message])
      }
    }
  }

  // Handle: startInspection, logout, getDealerCode,
  // isSDKInitialized, getSDKVersion, manualOfflineSync
}

extension ClearQuoteModule: ClearQuoteSDKDelegate {
  func inspectionCompletionStatus(
    identifier: String,
    message: String,
    code: Int,
    isOffline: Bool,
    serverQuoteId: String?,
    serverInspectionId: String?
  ) {
    guard hasListeners else { return }
```

```
    sendEvent(
      withName: "inspectionCompletionStatus",
      body: [
        "identifier": identifier,
        "message": message,
        "code": code,
        "isOffline": isOffline,
        "serverQuoteId": serverQuoteId ?? NSNull(),
        "serverInspectionId": serverInspectionId ?? NSNull(),
      ]
    )
  }
}
```

Step 2: Create ClearQuoteModule.m

Export the module with an Objective-C file. Do not put RCT_EXTERN_MODULE in the bridging header.

```
#import <React/RCTBridgeModule.h>
#import <React/RCTEventEmitter.h>

@interface RCT_EXTERN_MODULE(ClearQuoteModule, RCTEventEmitter)

RCT_EXTERN_METHOD(
  initSDK:(NSString *)key
  resolver:(RCTPromiseResolveBlock)resolve
  rejecter:(RCTPromiseRejectBlock)reject
)

RCT_EXTERN_METHOD(
  startInspection:(NSDictionary *)clientAttrs
  inputDetails:(NSDictionary *)inputDetails
  userFlowParams:(NSDictionary *)userFlowParams
  resolver:(RCTPromiseResolveBlock)resolve
  rejecter:(RCTPromiseRejectBlock)reject
)

RCT_EXTERN_METHOD(logout)
RCT_EXTERN_METHOD(manualOfflineSync)
RCT_EXTERN__BLOCKING_SYNCHRONOUS_METHOD(getDealerCode)
RCT_EXTERN__BLOCKING_SYNCHRONOUS_METHOD(getSDKVersion)
RCT_EXTERN__BLOCKING_SYNCHRONOUS_METHOD(isSDKInitialized)

@end
```

5.2 Android

Not Implemented yet

Android native bridge setup for ClearQuoteSDK is not available yet. Kotlin/Java native-module bridging steps will be documented when Android support is added.

6. React Native Usage

Create a JavaScript/TypeScript wrapper around the native module (for example src/ClearQuoteSDK.ts).

JS / TS wrapper

```
import { NativeEventEmitter, NativeModules } from 'react-native';

const { ClearQuoteModule } = NativeModules;

export class ClearQuoteSDK {
  static initSDK(key) {
    return ClearQuoteModule.initSDK(key);
  }

  static startInspection(clientAttrs, inputDetails, userFlowParams)
{
    return ClearQuoteModule.startInspection(
      clientAttrs ?? null,
      inputDetails ?? null,
      userFlowParams ?? null,
    );
  }

  static logout() {
    ClearQuoteModule.logout();
  }

  static getDealerCode() {
    return ClearQuoteModule.getDealerCode();
  }

  static isSDKInitialized() {
    return ClearQuoteModule.isSDKInitialized();
  }

  static manualOfflineSync() {
    ClearQuoteModule.manualOfflineSync();
  }
```

```
  static addInspectionCompletionListener(listener) {
    return new NativeEventEmitter(ClearQuoteModule).addListener(
      'inspectionCompletionStatus',
      listener,
    );
  }
}
```

Initialize SDK

```
const result = await ClearQuoteSDK.initSDK(sdkKey);
// Success when result.code === 200
```

Start Inspection

```
const result = await ClearQuoteSDK.startInspection(
  {
    userName: 'Jane',
    dealer: 'Dealer Name',
    dealerIdentifier: 'D123',
    client_unique_id: 'client-1',
    organisationId: 'org-1',
  },
  {
    customerDetails: {
      name: 'Customer',
      email: 'customer@example.com',
      dialCode: '+1',
      phoneNumber: '5550100',
    },
    vehicleDetails: {
      regNumber: 'ABC123',
      make: 'Toyota',
      model: 'Camry',
      bodyStyle: 'Sedan',
      fuelType: 'Petrol',
      variant: 'XLE',
    },
    quoteData: {
      inspectionType: 'standard',
      fleetImageType: null,
    },
  },
  {
    isOffline: false,
    skipInputPage: false,
  },
```

```
);
```

Listen for inspection completion

```
const sub = ClearQuoteSDK.addInspectionCompletionListener((status)
=> {
  // status: identifier, message, code, isOffline,
  //         serverQuoteId, serverInspectionId
});

// Cancel when done:
// sub.remove();
```

Logout / helpers

```
ClearQuoteSDK.logout();
const dealerCode = ClearQuoteSDK.getDealerCode();
const initialized = ClearQuoteSDK.isSDKInitialized();
ClearQuoteSDK.manualOfflineSync();
```

7. Native Module API

Native module: ClearQuoteModule

- initSDK — args: key. Returns { isInitialized, code, message }
- startInspection — args: { clientAttrs, inputDetails, userFlowParams }. Returns { started, message, code }
- logout — clears SDK session
- getDealerCode — returns String?
- isSDKInitialized — returns bool
- manualOfflineSync — triggers offline inspections sync

More public methods/properties are available in SDK which can be found in below docs:

https://docs.google.com/document/d/1eqHUg3L7mqA4E8vqslzpLoqoC_8qxv7wTUn_JneKQmY/edit?usp=sharing

https://docs.google.com/document/d/1qaoIRasNhM7pLG6hKX2aLnKaZr35R-_8GSMnZpDO9Sw/edit?usp=sharing

Event: inspectionCompletionStatus

Payload fields:

- identifier, message, code, isOffline, serverQuoteId, serverInspectionId

8. Error Codes (Initialization)

Possible messages from ClearQuote SDK while initializing:

- “SDK key is empty” — Recheck if the SDK key is passed correctly
- “Invalid user region” — Recheck if the SDK key is passed correctly
- “SDK Internal Error” — Contact ClearQuote team
- “Success” — ClearQuote SDK initialized successfully

Possible error codes:

- 101 — SDK key is empty
- 100 — Invalid user region
- 500 — SDK Internal Error
- 200 — Success

9. Notes

- Do not initialize the SDK multiple times. Use isSDKInitialized() if required
- Camera and location permissions are mandatory for inspection flows
- Always invoke SDK UI methods on the main thread (the iOS bridge does this)
- Open xcworkspace of your project when working in Xcode so SPM packages resolve correctly

10. Demo App

Below repo is a sample React Native app with ClearQuoteSDK integration

https://github.com/clearquotetech/react-native-sample-project

11. Support

If you face issues, raise a GitHub issue with logs and screenshots:

https://github.com/clearquotetech/cq-ios-sdk/issues

For queries:

- sharath@clearquote.io
- bhanusingh@clearquote.io
- akhila@clearquote.io